获NVIDIA致谢：悟空Agent的实践、复盘与迭代

腾讯技术工程 2025年11月7日 01:37 广东 492人



作者：悟空代码安全团队

> 导语： 当代码复杂度爆炸，如何让代码安全审计回归高效与理性？本文以悟空 Agent 获得NVIDIA官方致谢为起点，介绍多智能体架构如何闭环，以及在实战中面临的真实挑战——上下文断流、调度失衡。这是一场务实的探索：让 AI 成为安全团队的第二双眼，让安全真正融入业务生产线。

一、人类，已经看不完代码了

> AI 的意义不在于取代，而在于放大人类的判断力。它让代码安全审计重新回归“人”的部分——分析、推理、决策。

代码安全审计，起初是依靠安全专家经验与直觉的“手艺”，后来随着“手艺人”越来越多，经验逐渐被沉淀、规则化，于是诞生了 SAST（静态应用安全测试）工具。

无论是靠经验，还是靠规则，都曾是代码安全领域最倚重的报警器。

但当 AI 时代来临，项目规模动辄数十万文件、数百万行代码时，这两道报警器都出现了同一个问题：**告警的速度赶不上代码复杂度。**



审计方式	优点	核心痛点	典型后果
人工审计	能深入理解业务逻辑 能识别复杂漏洞链	人工成本高、周期长 依赖专家经验与直觉	审计周期以周计 主观差异，易疲劳遗漏
SAST工具	能批量扫描常见漏洞（XSS、SQLi）	误报率高 上下文缺失、规则更新慢	告警的“噪音”过多 结果仍需人工复核

人工审计就像手术刀，虽精准但稀缺，而SAST像扫描仪，方便快捷却不治根。

二者结合也能覆盖更多场景，却依旧难以应对 AI 时代里代码的迭代速度。像 LangChain、LLaMA-Factory、LLaMA-Index 这些开源明星 AI 框架，几乎每周都有新模块与依赖更新。其代码中任何一个 yaml.load() 调用、一次 checkpoint 加载，都随时可能成为潜在的漏洞爆发点。

正因如此，悟空代码安全团队一直在探索一种新的思路——不是替代人，而是让机器分担那些繁复却关键的逻辑工作。

二、一次来自NVIDIA的致谢

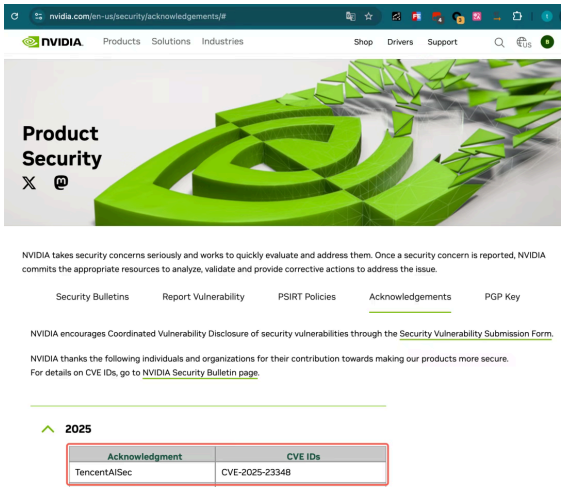
在一次常规的 AI 框架安全审计中，悟空 Agent 注意到 NVIDIA Megatron-LM 项目中的一个反序列化逻辑存在潜在风险。

pretrain_gpt.py 文件在加载 YAML 配置时使用了不安全的反序列化方式，攻击者理论上可通过恶意输入触发命令执行。经过安全专家的复现确认，旧版 PyYAML（<5.4.1）在处理 yaml.load() 调用时，会直接执行对象实例化逻辑，从而引发命令执行。

修复方式简单且有效：将 yaml.load() 替换为更安全的 yaml.safe_load() 即可。

在2025年9月，NVIDIA 发布了官方安全公告：

> Security Bulletin: NVIDIA Megatron-LM - September 2025
> CVE-2025-23348 — “Malicious data in pretrain_gpt may cause code injection, leading to code execution, privilege escalation, and data tampering.” Severity: High (CVSS 7.8)
>
> 来源：NVIDIA Official Security Bulletin - September 2025



公告同步修复了该问题，并在版本 0.13.1 / 0.12.3 中完成更新。

短短一句致谢背后，是一次完整的安全协作闭环 —— 从漏洞发现、复现验证、负责任披露，到官方确认、修复发布。

三、 悟空 Agent 的架构设计与迭代效率

> 悟空 Agent 的核心不是“模型大”，而是“迭代快”。 我们致力于让每一个误报、漏报、复测都能成为下一个版本的训练素材。

当漏洞发现能力不再取决于单一模型性能，而取决于“体系闭环”的成熟度时，我们开始将注意力转向架构设计以及数据工程飞轮的构建。

1. 架构：协同作战的智能军团

我们将代码审计任务拆解为多智能体（Multi-Agent）的协作流程，这一设计的理念是：专家管方向，Agent管执行。

在实际工程中，以五个智能体完整闭环：

角色	职责定位	核心价值

Client Agent	用户交互入口，接收任务、生成配置、反馈结果	把复杂任务标准化、可批量分发
Remote Agent	负责任务规划与路由，将复杂审计拆解为可执行子任务	提升整体调度效率，避免重复扫描
Audit Agent	漏洞挖掘核心单元，对代码片段和完整项目进行多层次扫描与风险评估	覆盖广度与深度，发现高复杂漏洞链
Review Agent	复审检测结果，结合多 Prompt 和评分机制确认漏洞有效性	降低误报率、稳定输出结果质量
Fix Agent	生成修复方案，输出安全建议并验证补丁有效性	实现漏洞到修复的全闭环

在对开源项目的持续测试中，我们看到这样的协同带来了明显提效：

● 代码扫描与路径分析的耗时缩短约 80%；

● 误报率下降超过 60%；

> 悟空 Agent 架构、流程图详见：0day漏洞量产？AI Agent“生产线”曝光

这并不是“效率革命”，而是一次很务实的调整，把有限的安全专家人力投入在最值得分析与决策的地方。

2. 复盘：Agent迭代中的瓶颈与优化

在持续的版本迭代中，我们发现多智能体协作虽显著提升了效率，但也暴露出一些典型问题。这些问题集中在两个层面：准确率瓶颈、任务调度不均衡。

(1) 准确率的瓶颈：上下文断流

在使用悟空 Agent 进行项目级扫描时，我们发现漏洞的准确率高度依赖上下文信息。而在复杂代码项目中，大模型往往无法完整捕获输入输出的上下文链，造成“断流”现象。我们将原因归纳为两点：

● 框架复杂性导致上下文提取不足。 多层封装、动态 import、异步逻辑等模式，使得调用链在语义层面被“截断”；

● 内部特定代码用法缺乏知识覆盖。 某些自研框架或宏定义逻辑不在开源语料中，模型难以正确解析。

针对这两类问题，悟空 Agent 团队引入了系统性的“上下文工程（Context Engineering）”方案：

i）增强上下文提取：

通过专用上下文提取工具，针对类继承、装饰器逻辑、框架 Hook 等结构设定提取模板，实现多层语义补全。

ii）知识补全机制：

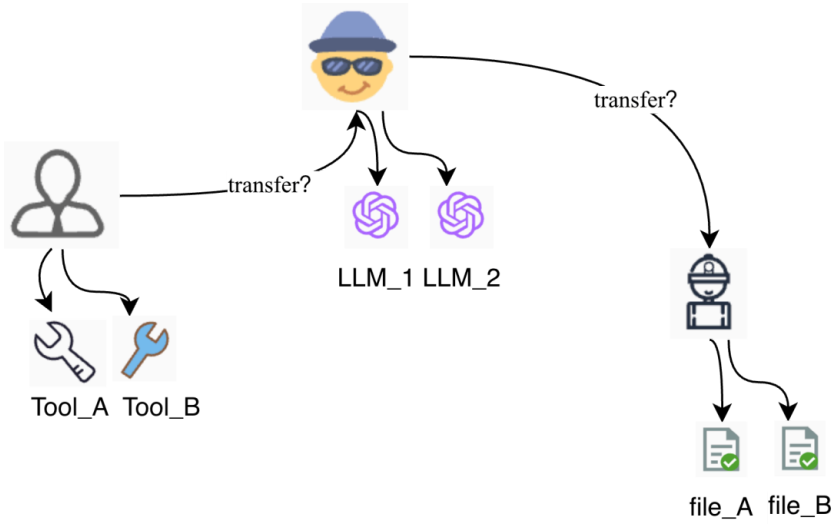
在识别到“内部专用调用”时触发 RAG-like 机制，从内部知识库实时检索相应逻辑并注入上下文，使模型具备“局部知识回忆”能力。

经过持续优化后，Bad Case的归因命中率再次提升——多数误报都可在第二轮迭代中被消化。

（2）智能体调度失衡：协作退化为串行

在多智能体架构下，理论上任务应在多个 Agent 间分工协作，但在实际运行中，我们发现：

即使定义了各自Agent的职责与工具集，在解决用户需求的时候多个 Agent 仍倾向于让“主Agent”独自处理任务。而不是将任务分发给不同职能的其他智能体来解决。



这就带来了一个问题，原本是为了区分独立的上下文，从而提升子任务的解决效果而设计的多智能体架构，退化为串行工具调用的单智能体架构，导致系统整体表现下降。

在进行大量调研后，我们后续计划通过强化学习（RL）调度机制解决该问题：

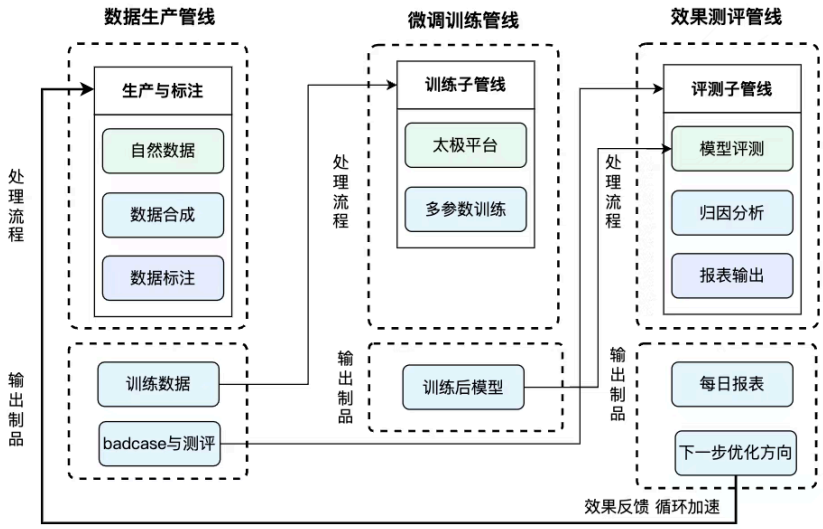
- 采集少量高质量标注样本，训练模型识别任务场景与角色最优分配；
- 让大模型学会在不同上下文中主动“移交任务”，而非被动等待指令；

3. 数据飞轮：从Bad Case到模型迭代

这套多 Agent 架构的背后，是一个能持续“自我进化”的 数据飞轮体系。

系统每天从真实扫描场景中抽取 Bad Case 样本，经模型判断和人工质检后进入训练与评测管线。所有误报、漏报都会被分类入库，用于后续微调与算法优化。

我们的数据飞轮体系有三条管线：

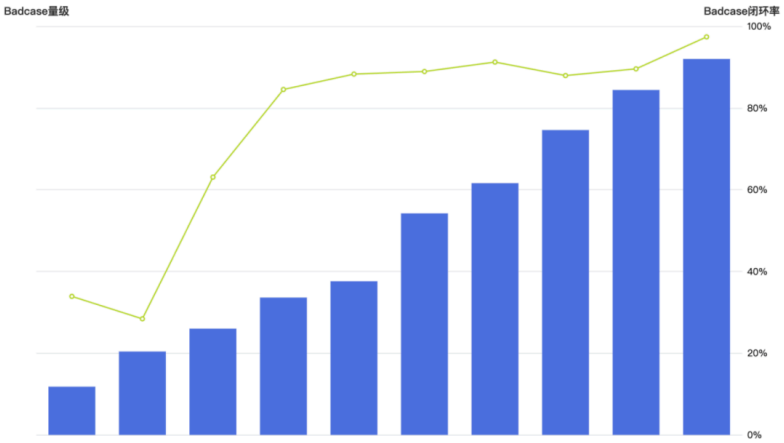


管线	主要功能	输出
数据生产管线	收集 Bad Case 与合成数据，经模型判断与质量评估后入库	评测集、微调数据
微调训练管线	一站式模型调用与多参训练，自动生成 CoT 推理链	微调模型版本
效果评测管线	模型评测、归因分析、报告生成，形成每日 Bad Case 追踪	优化方向与迭代计划

系统每日从实际扫描输出的Bad Case中抽取代表性样本，经模型复核与人工质检后进入训练集。

所有模型评测结果都会被自动归因并量化回流，形成优化建议。

经过数轮版本迭代，Bad Case 的闭环解决率已从最初的 30% 提升至 97% 以上，绝大多数问题都能在数据飞轮体系内被消化，而不再依赖大量人工调试。



四、 AI Infra的现实：更多细微的安全裂缝

在与多个开源社区的协作过程中，我们越来越清楚地看到一个事实——AI infra 并不天然安全。模型越来越强，代码越来越复杂，但底层的工程基础，却常常建立在“信任默认值”之上。在我们的日常审计中，这些“默认”几乎随处可见：

➢ 配置文件没有边界校验、序列化机制依旧沿用早期模板、外部依赖包随版本漂移、训练脚本混合使用多个框架……

在复杂的 AI 流水线中，这些细节并不会立即出错，却足以让一个成熟系统暴露出可被利用的攻击面。有些安全研究人员认为 AI 安全问题主要集中在模型本身，但现实告诉我们——真正的风险往往在模型之外：

➢ 它可能是一段被沿用的 YAML 加载逻辑、一个疏忽的 WebUI 参数解析，也可能是某个未更新的依赖版本。

这不是危言耸听，而是我们在主流框架中一次次验证过的事实：

● LLaMA-Factory 的模型加载链：悟空Agent实战：LLaMA-Factory高危0day漏洞挖掘与修复

● MS-SWIFT 框架的命令拼接点;

● LangChain 的默认启用组件EverNoteLoader;

● ···

每一次发现都让我们意识到：AI 基础设施的发展速度，远远快于安全体系的适配速度。

每一次修复背后，都是一次“安全假设”被重新检验的过程。这些漏洞不只是代码问题，更像是一面镜子，时刻提醒我们，在这场 AI 浪潮中，每一次加速都必须伴随一次“回头看”：

那些连接一切的 AI 基础设施，是否真的安全?

尾声：当安全回归业务战场和“人”

当技术、业务与信任汇聚成同一个目标时，安全，终于不再只是成本，而是一种长期的竞争

力。

悟空 Agent 的所有研究与实战，并不是为了展示模型的能力，而是为了让这些能力，真正融入业务生产线。过去几个月里，我们与微信、广告等业务上，和业务安全团队一起共建代码审计与模型防护体系，也产出了一些成果。

无论是客户端的 RCE 漏洞挖掘，还是业务的接口安全与策略验证，这些看似隐形的审计与修复，正在一点点减少潜在公司业务面临的风险、提升系统韧性，让创新更可持续。

AI 正在改变我们构建产品的方式，也在重新定义“安全”这件事的边界。

在这场变革中，我们逐渐意识到：

真正的安全，不只是防御漏洞，更是让业务、算法与信任可以共同成长的能力。

AI Agent 帮我们分担了疲惫的机械环节，让安全团队有更多精力去思考“为什么这样设计”，而不是“哪里出了错”。它的意义，不在于自动化本身，而在于让安全真正成为业务的一部分，而非阻力。

我们期待一个新的安全范式：在那里，AI 不仅是被保护的对象，也能成为守护者；安全不再是被动防御，而是业务创新的底层支撑。这才是人机协同的真正价值。





上一篇
0day漏洞量产？AI Agent“生产线”曝光

下一篇
首发！建议你一定要看的《AI 生成代码在野安全风险研究》

阅读 1.2万

留言

写留言

还没有留言，来写首评